



Лабораторийн ажил 10

Загвар класс

Мэдээлэл, Компьютерын Ухааны тэнхим

Хичээлийн нэр (индекс): Объект хандлагат программчлал (ISCI201)

Лабораторийн хуваарь: Пүрэв гариг 07:40 – 10:05 цаг

Суралцагчдийн нэр: Э.Баттамир, Ц.Сумьяабазар, Г.Билгүүн

Багшийн нэр: М.Золжаргал (Ph.D)

2026.05.14 | МУИС хичээлийн 8-р байр

Загвар класс гэж юу вэ?

Загвар буюу ерөнхий өгөдлийн төрөл гэдэг нь параметржсэн өгөдлийн төрлийг хэлнэ. Энэ хэлбэрээр дамжуулан өгөгдлийн төрлийг өөрчлөх боломжтой.

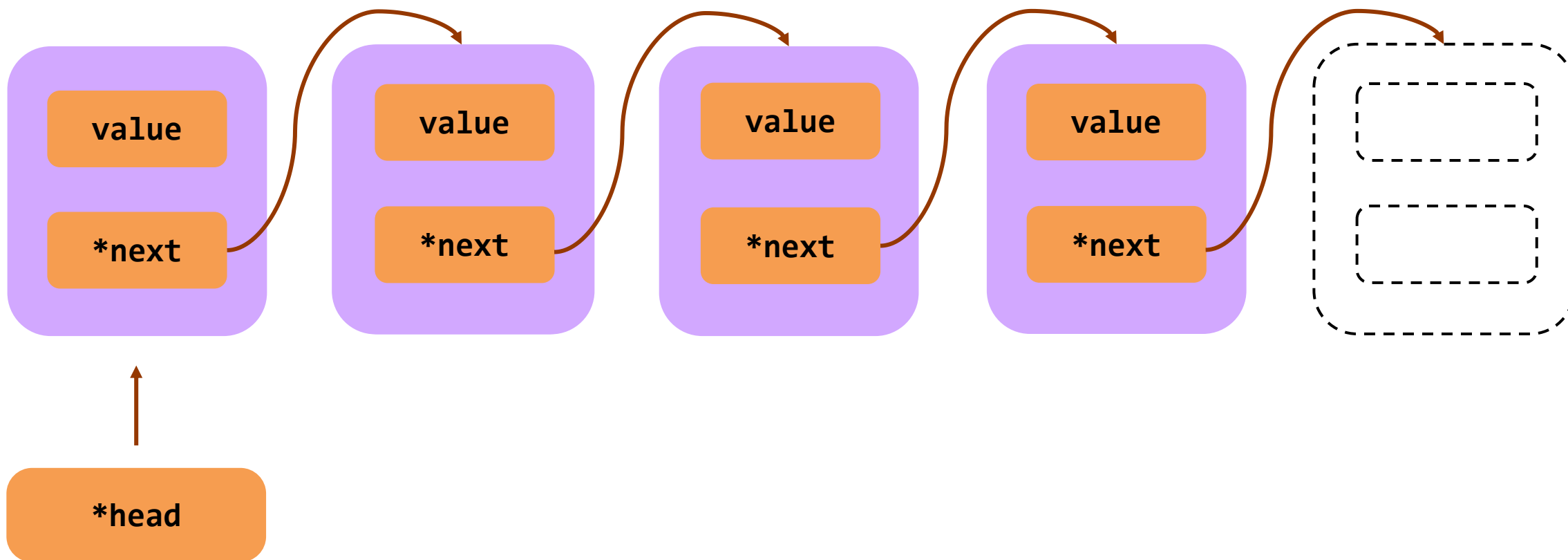
Template (загвар) нь төрөл үл хамаарах програмчлал хийхэд зориулагдсан.

Template ашигласнаар:

- Код давтаж бичихгүй,
- Аль ч төрлийн өгөгдөл дээр ажилладаг нэг л класс үүсгэнэ,
- Компиллах явцад T нь бодит төрлөөр солигдож ажилладаг.

Өөрөөр хэлбэл, бид төрөл бүрийн объектуудыг (жишээ нь, Circle, Square, Triangle) нэг төрлийн загвараар хадгалах, удирдах боломжтой болно.

Холбоост жагсаалт гэж юу вэ?



Хэрэгжүүлэлт: List класс

```
template <typename T>
class List {
    private:
        Node<T>* head;
        int len;
    public:
        List(): head(nullptr), len(0) {}
        void add (T t);
        void insert(T t, int index);
        Node<T>* getHead();
        T get(int index);
        void deleteNode(int index);
        int length();
};
```

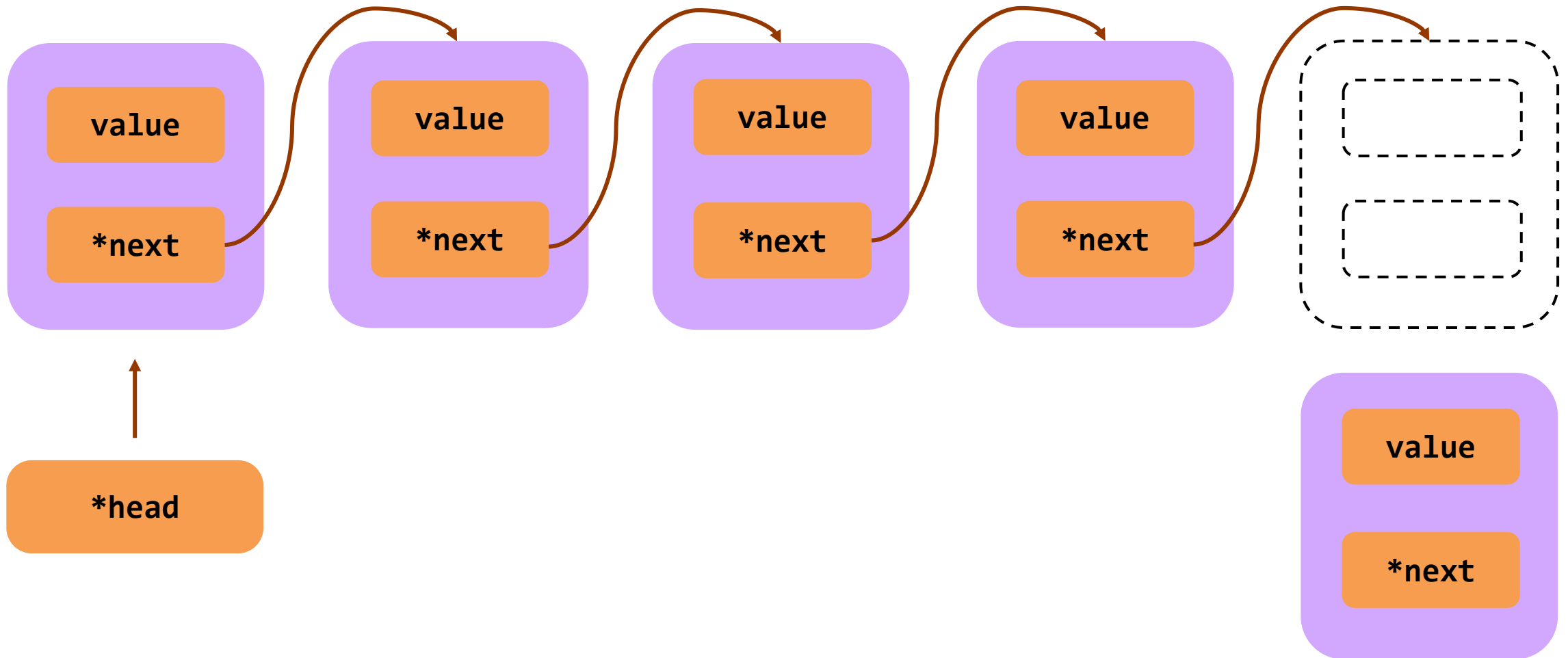
```
template <typename T>
struct Node {
    T val;
    Node* next;
};
```

Хэрэгжүүлэлт: add() гишүүн функц

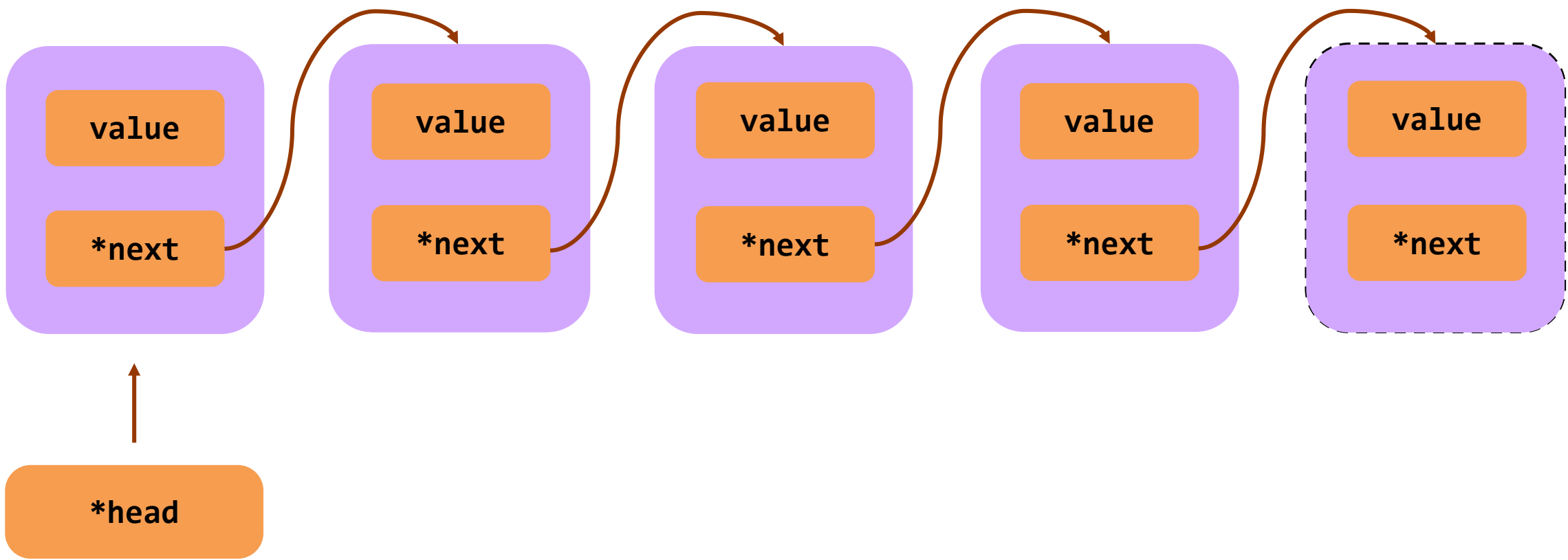
```
template <typename T>
void List<T>::add(T t)
{
    Node<T>* newNode = new Node<T>();
    newNode->val = t;
    newNode->next = nullptr;

    if (head == nullptr) head = newNode;
    else {
        Node<T>* temp = head;
        while (temp->next != nullptr) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
    len++;
}
```

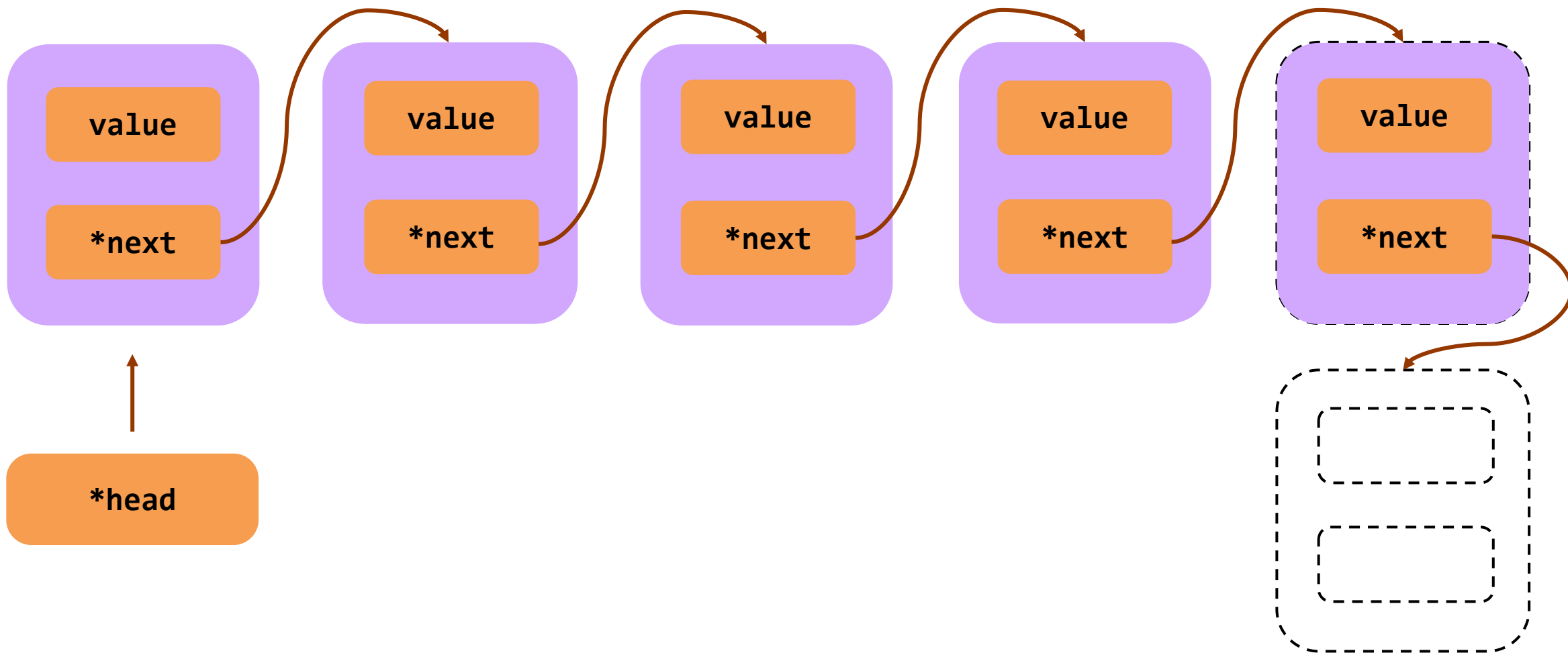
Хэрэгжүүлэлт: add() гишүүн функц



Хэрэгжүүлэлт: add() гишүүн функц



Хэрэгжүүлэлт: add() гишүүн функц



Хэрэгжүүлэлт: deleteNode() гишүүн функц

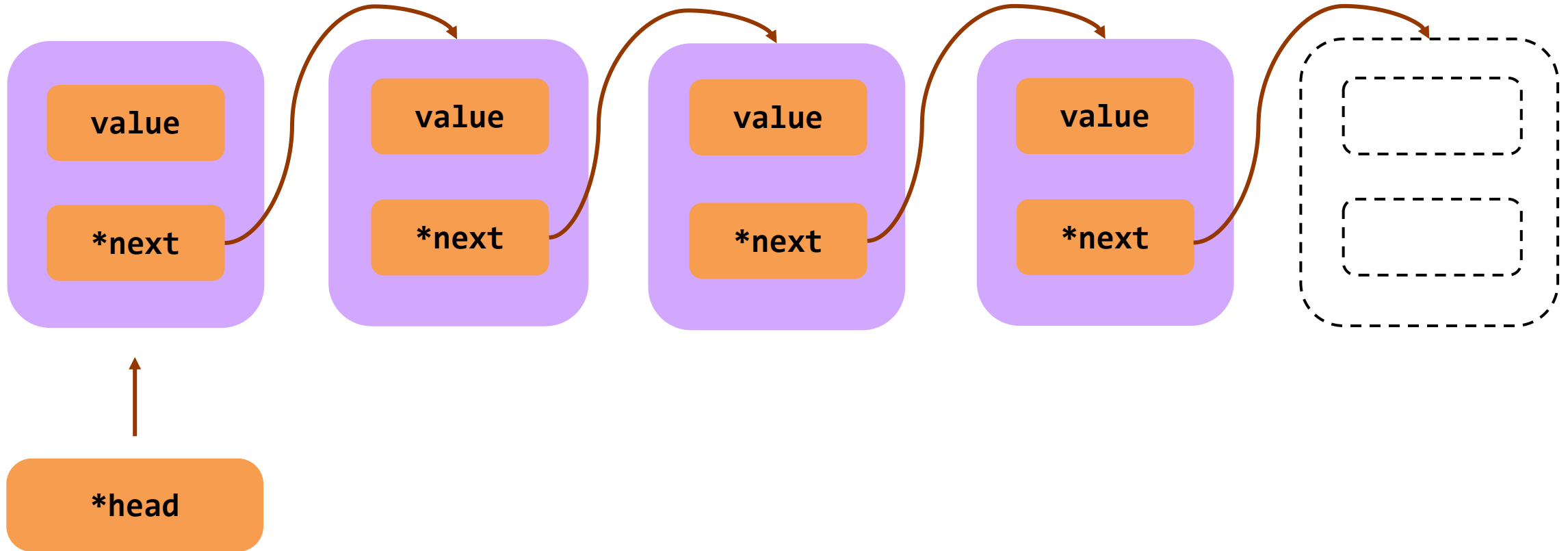
```
template <typename T>
void List<T>::deleteNode(int index)
{
    if (head == nullptr || index < 0) return;
    Node<T>* temp = head;
    if (index == 0) {
        head = head->next;
        delete temp;
        len--;
        return;
    }

    Node<T>* prev = nullptr;
    for (int i = 0; i < index && temp != nullptr; i++) {
        prev = temp;
        temp = temp->next;
    }

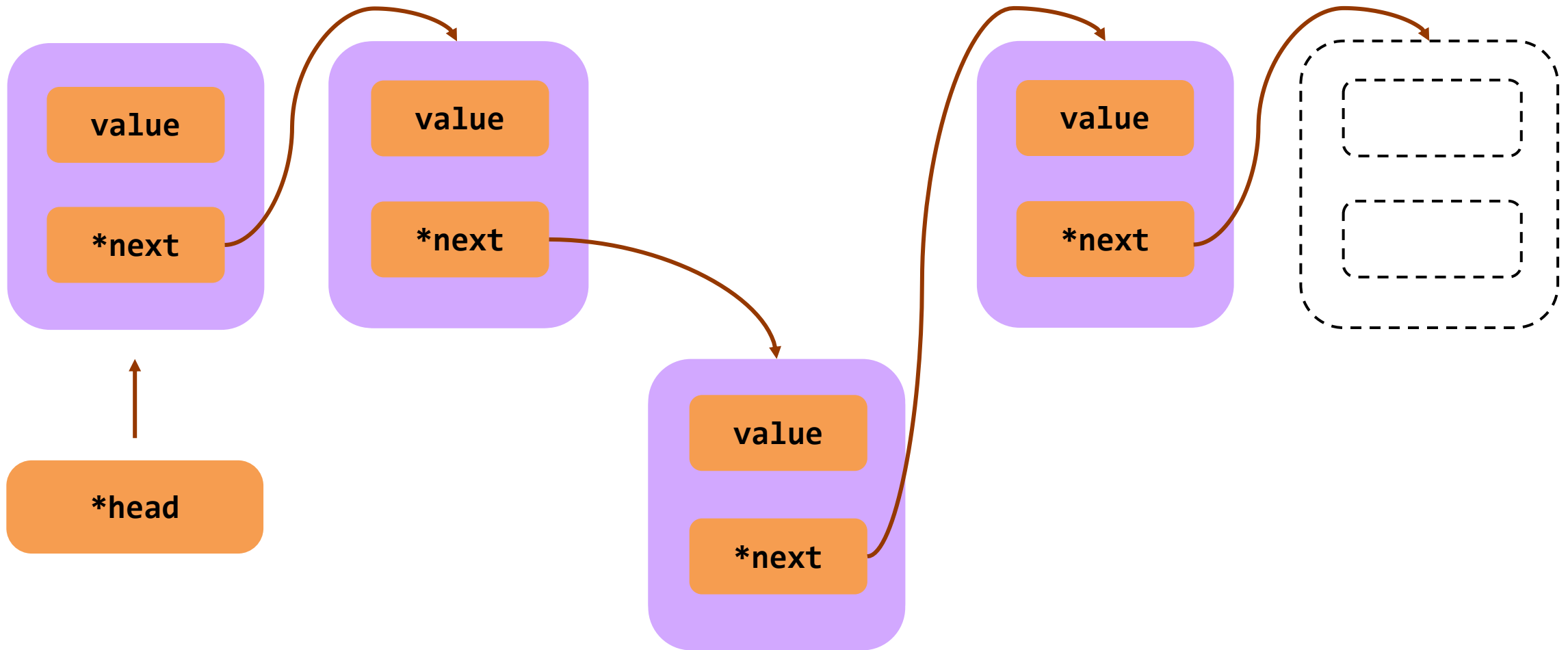
    if (temp == nullptr) {
        return;
    }

    prev->next = temp->next;
    delete temp;
    len--;
}
```

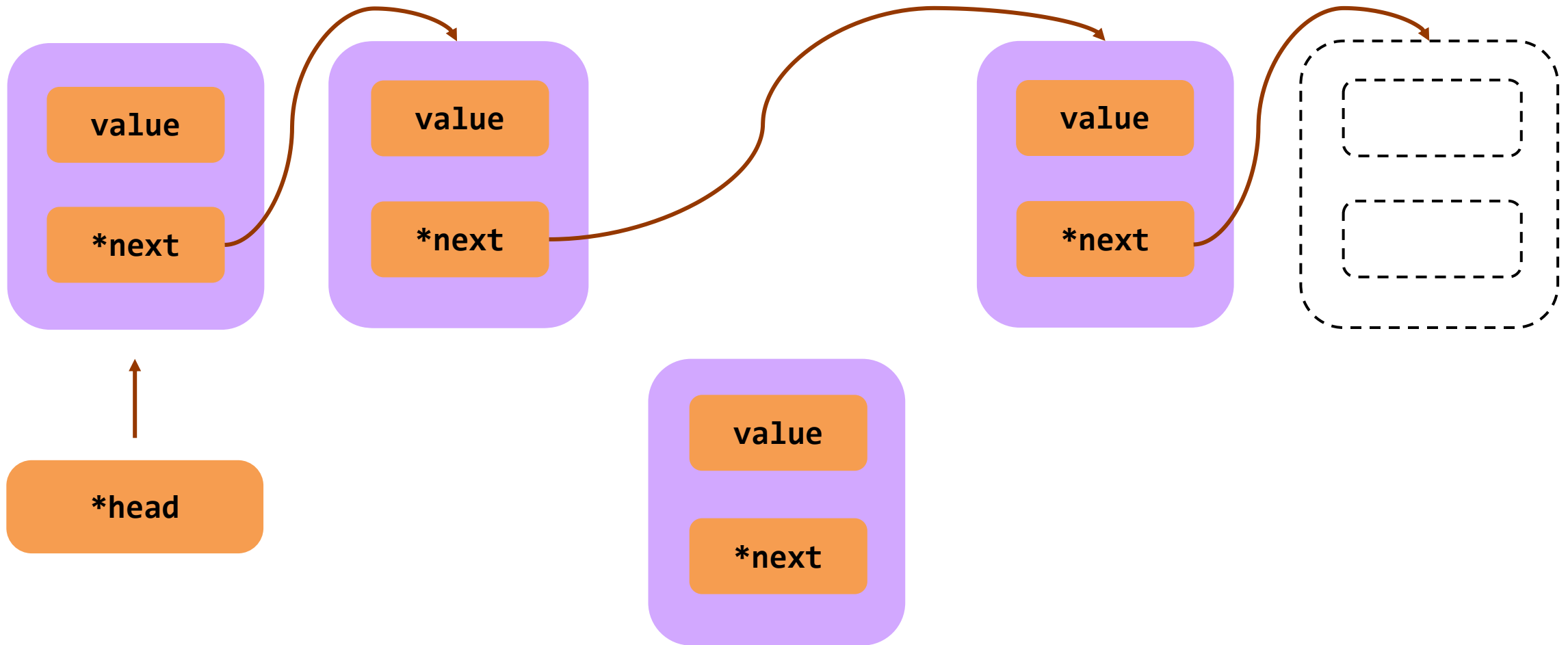
Хэрэгжүүлэлт: `deleteNode()` гишүүн функц



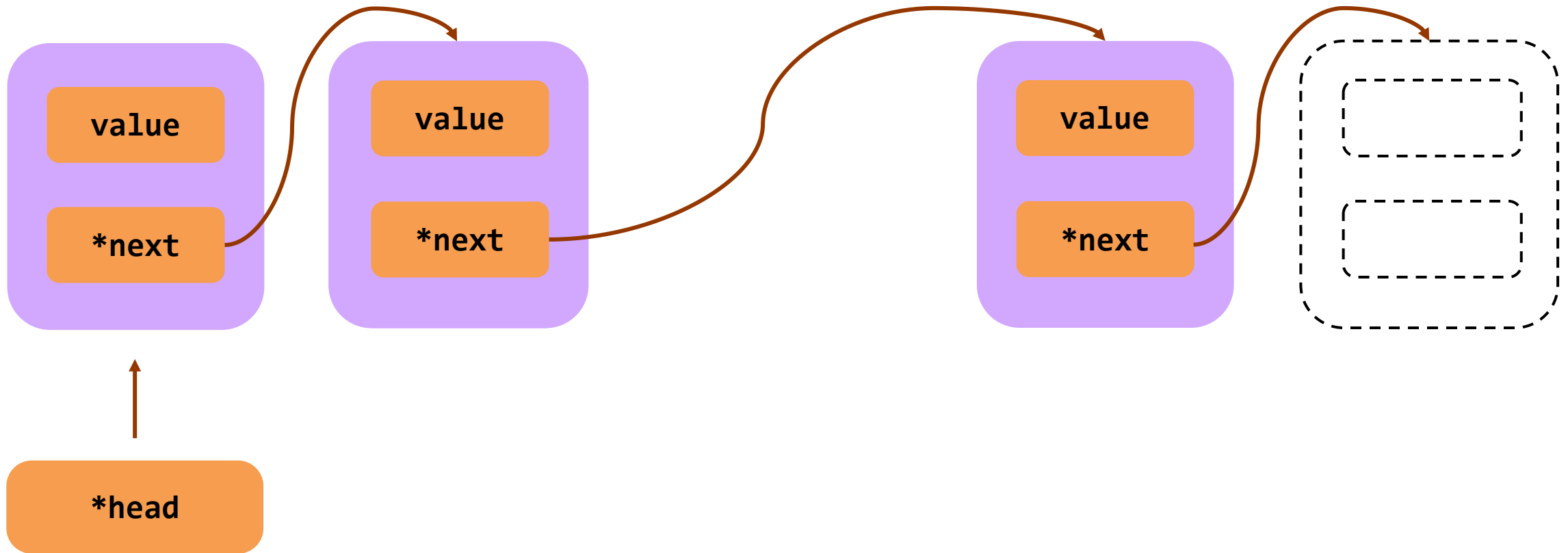
Хэрэгжүүлэлт: `deleteNode()` гишүүн функц



Хэрэгжүүлэлт: deleteNode() гишүүн функц



Хэрэгжүүлэлт: `deleteNode()` гишүүн функц



Хэрэгжүүлэлт: insert() гишүүн функц

```
template <typename T>
void List<T>::insert(T t, int index) {
    if (index < 0 || index > len) return;

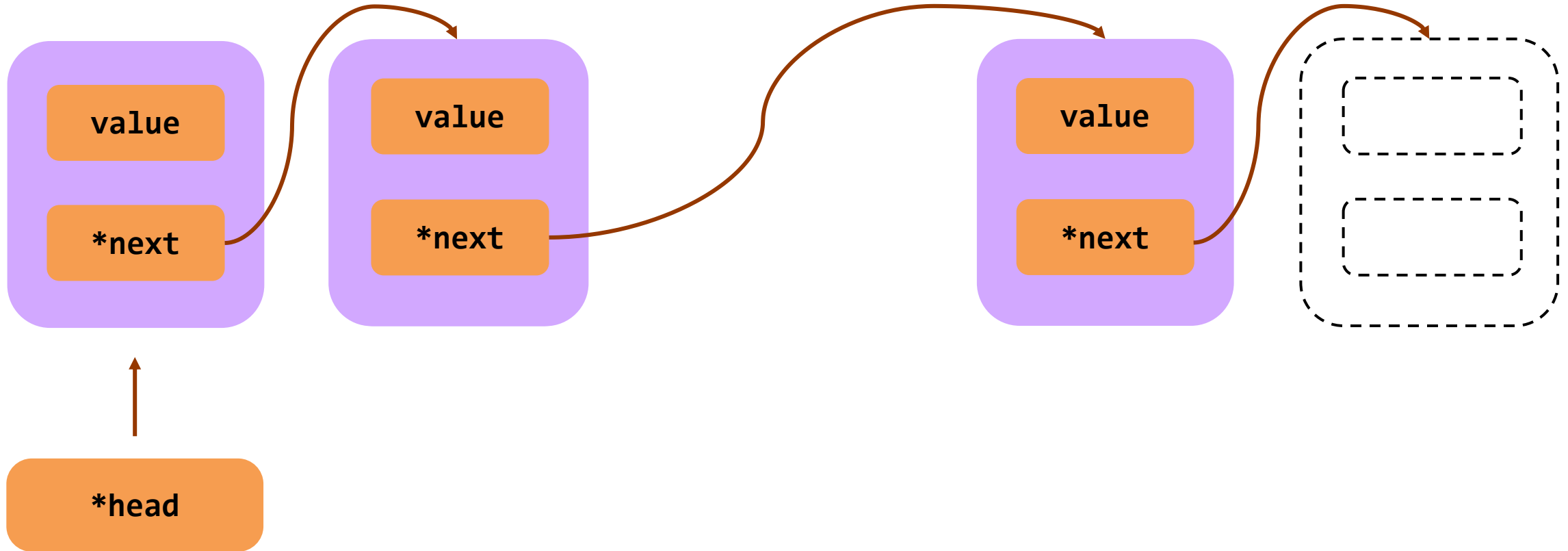
    Node<T>* newNode = new Node<T>();
    newNode->val = t;
    newNode->next = nullptr;

    if (index == 0) {
        newNode->next = head;
        head = newNode;
    }
    else {
        Node<T>* temp = head;
        for (int i = 0; i < index - 1; i++) {
            temp = temp->next;
        }

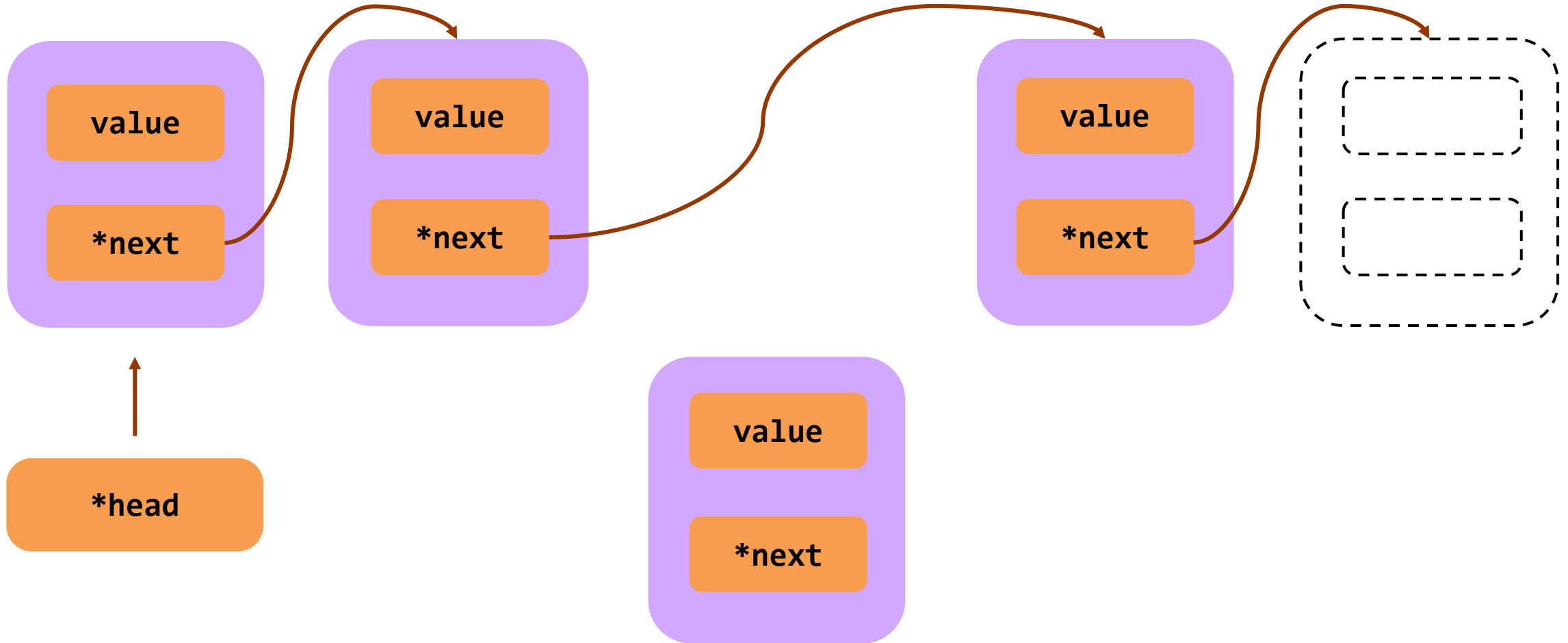
        newNode->next = temp->next;
        temp->next = newNode;
    }

    len++;
}
```

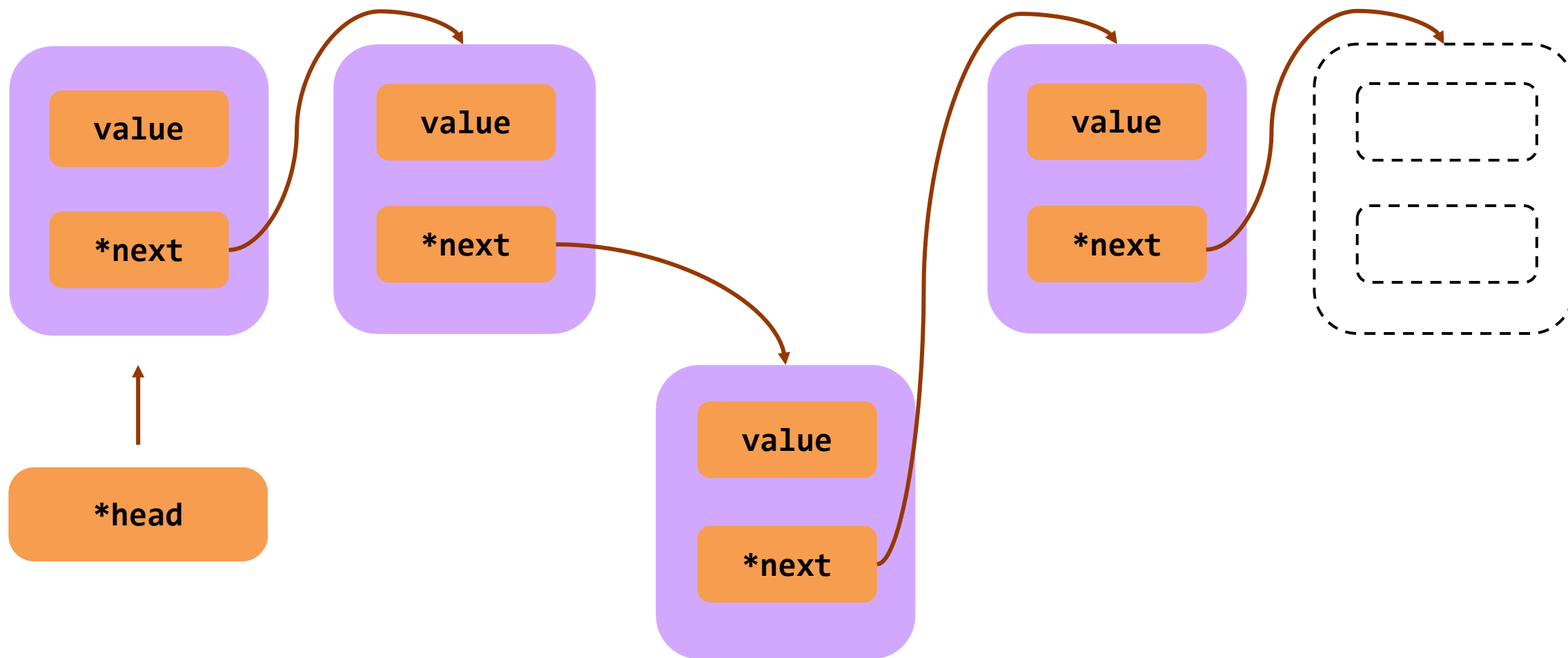
Хэрэгжүүлэлт: `insert()` гишүүн функц



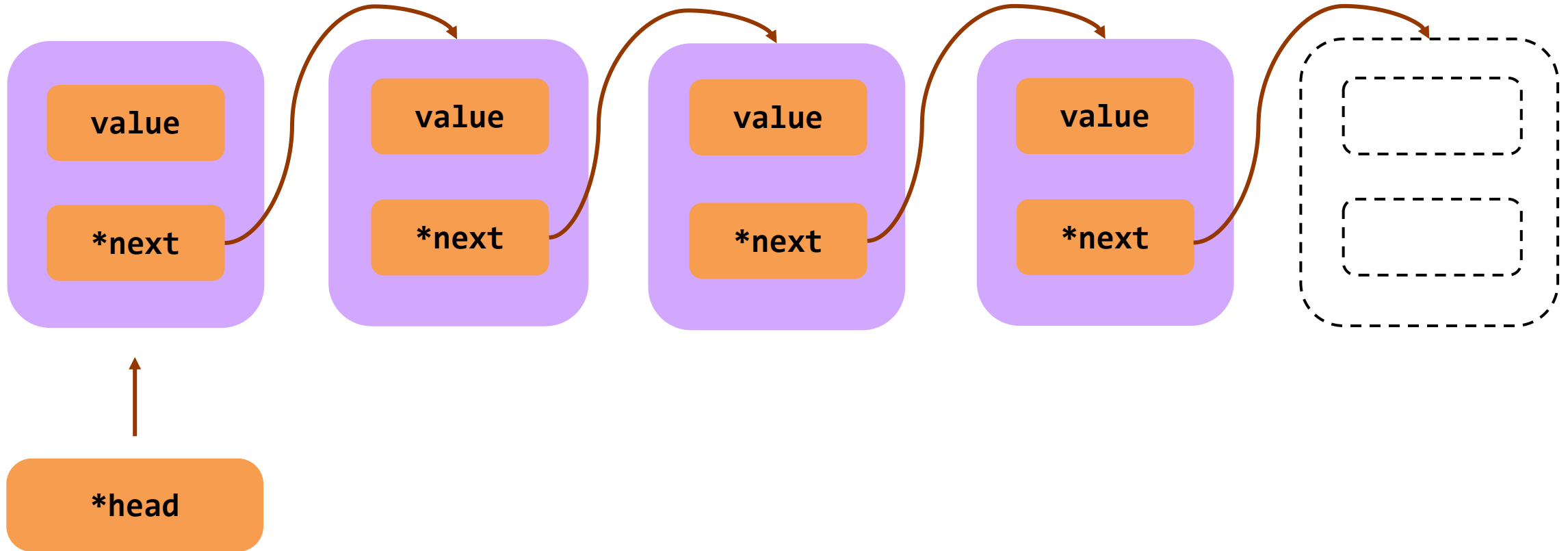
Хэрэгжүүлэлт: `insert()` гишүүн функц



Хэрэгжүүлэлт: `insert()` гишүүн функц



Хэрэгжүүлэлт: `insert()` гишүүн функц



Хэрэгжүүлэлт: get() гишүүн функц

```
template <typename T>
T List<T>::get(int index) {

    if (index < 0 || index >= len) {
        throw out_of_range("Index out of range");
    }

    Node<T>* temp = head;

    for (int i = 0; i < index; i++) {
        temp = temp->next;
    }

    return temp->val;
}
```

Хэрэгжүүлэлт: length() гишүүн функц

```
template <typename T>  
int List<T>::length()  
{  
    return len;  
}
```

Хэрэгжүүлэлт: makeRandoms() функц

```
#include "functions.hpp"
```

```
List<Shape2D*> makeRandoms() {  
    random_device rd;  
    mt19937 gen(rd());  
    uniform_real_distribution<> real(1.0, 100.0);  
    uniform_int_distribution<> mySwitch(1, 3);  
    uniform_int_distribution<> many(20, 30);  
  
    cout << "Making random shapes..." << endl << endl;  
  
    List<Shape2D*> shapeList;  
    int count = many(gen);
```

Хэрэгжүүлэлт: makeRandoms() функц

```
for (int i = 0; i < count; i++) {  
    int k = mySwitch(gen);  
    double what = real(gen);  
  
    if (k == 1) {  
        shapeList.add(new Square(what, 0, 0));  
    } else if (k == 2) {  
        shapeList.add(new Circle(what, 0, 0));  
    } else {  
        shapeList.add(new Triangle(what, 0, 0));  
    }  
}  
return shapeList;  
}
```

Хэрэгжүүлэлт: print() функц

```
void print(List<Shape2D*> &shapeList)
{
    Node<Shape2D*> *curr = shapeList.getHead();

    int count = 1;
    while (curr)
    {
        Shape2D* shape = curr->val;
        if (shape) {

            cout << count << ".\t" << "Shape: " << shape->getName() << "\t\t"
                 << "Area: " << shape->area() << endl;

        }
        curr = curr->next;
        count++;
    }
}
```

Хэрэгжүүлэлт: main

```
#include "oop.hpp"

int main() {
    List<Shape2D*> shapeList = makeRandoms();
    sortByArea(shapeList);
    print(shapeList);
    clearShapes(shapeList);

    return 0;
}
```


Хэрэгжүүлэлт: Программын гаралт

```
defint@defint-Inspiron-15-3530:~/num/oop/oop/lab10$ ./go
Making random shapes...
```

```
24 objects sorting...
```

1.	Shape: Triangle	Area: 3.66926
2.	Shape: Circle	Area: 18.3284
3.	Shape: Triangle	Area: 23.2609
4.	Shape: Triangle	Area: 55.0408
5.	Shape: Square	Area: 100.038
6.	Shape: Circle	Area: 272.953
7.	Shape: Triangle	Area: 405.632
8.	Shape: Square	Area: 1274.69
9.	Shape: Circle	Area: 1526.66
10.	Shape: Square	Area: 1596.35
11.	Shape: Square	Area: 1679.59
12.	Shape: Circle	Area: 3043.51
13.	Shape: Square	Area: 3362.64
14.	Shape: Circle	Area: 3386.6
15.	Shape: Square	Area: 3765.1
16.	Shape: Triangle	Area: 3957.59
17.	Shape: Square	Area: 4831.85
18.	Shape: Square	Area: 6400.57
19.	Shape: Square	Area: 7716.48
20.	Shape: Square	Area: 9934.01
21.	Shape: Circle	Area: 13963.1
22.	Shape: Circle	Area: 19356.9
23.	Shape: Circle	Area: 25698.7
24.	Shape: Circle	Area: 31053.8

```
Deletting shapes...
```

```
Done.
```

Дүгнэлт

Бид энэхүү лабораторийн ажлаар C++ хэлний **Template** буюу загвар класс-ыг ашиглан дурын өгөгдлийн төрөлд ажиллах чадвартай **Холбоост жагсаалт (Linked List)**-ыг хэрэгжүүллээ. Үүний үр дүнд:Өгөгдлийн төрлөөс үл хамааран ажилладаг уян хатан код бичиж сурсан.

Жагсаалтад элемент нэмэх, устгах, индексээр хандах зэрэг үндсэн функцуудыг загвар класс дотор амжилттай тодорхойлсон.

Мөн өмнөх хичээлээр үзсэн дүрсийн классуудыг энэхүү жагсаалттайгаа холбож, олон төрлийн объектуудыг (гурвалжин, дөрвөлжин, тойрог) нэг бүтцэд хадгалан, талбайгаар нь эрэмбэлэх бодит хэрэглээг туршиж үзлээ. Энэхүү ажил нь төрөл үл хамаарах програмчлалын давуу талыг практик дээр ойлгоход ихээхэн тус боллоо.

Эх сурвалж

1. М. Золжаргал багшийн лекцийн материал
2. Объект хандлагат технологийн C++ программчал, Ж. Пүрэв, 2008, Улаанбаатар
3. [geeksforgeeks](http://www.geeksforgeeks.com)

**Анхаарал хандуулсанд
баярлалаа**